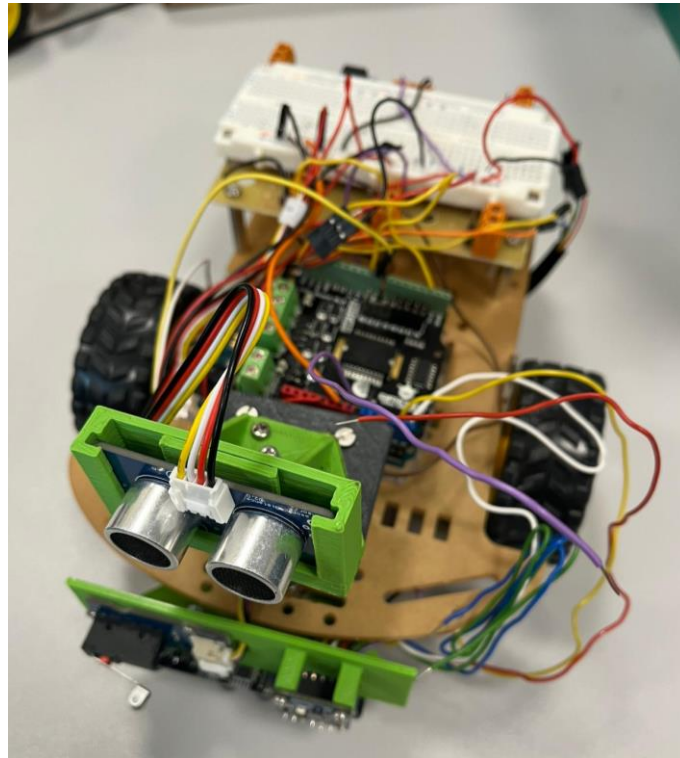




1. Introduction

Le projet s'inscrit dans la continuité de celui du semestre précédent : En plus du suivi de ligne, le robot doit être capable d'exécuter plusieurs tâches. Pour cela, le robot mobile sera équipé de capteurs numériques et/ou analogiques.

Certains capteurs sont déjà montés sur le robot. Nous avons également à notre disposition plusieurs autres capteurs, tels que des capteurs de couleurs, des télémètres, etc., que nous allons embarquer sur le robot. Nous sommes donc

amenés à récupérer l'information analogique ou numérique, image de la grandeur physique, de ces capteurs. Les informations récupérées sur la carte à microcontrôleur (ici un Atmega328P) seront traitées pour réaliser les différentes tâches répondantes au scénario.

Le robot mobile doit réaliser une succession de tâches depuis un point de départ jusqu'au point d'arrivée. Nous verrons par la suite de ce rapport en détail les étapes de chaque scénario tout en présentant des solutions trouvées pour chacune.

Sommaire

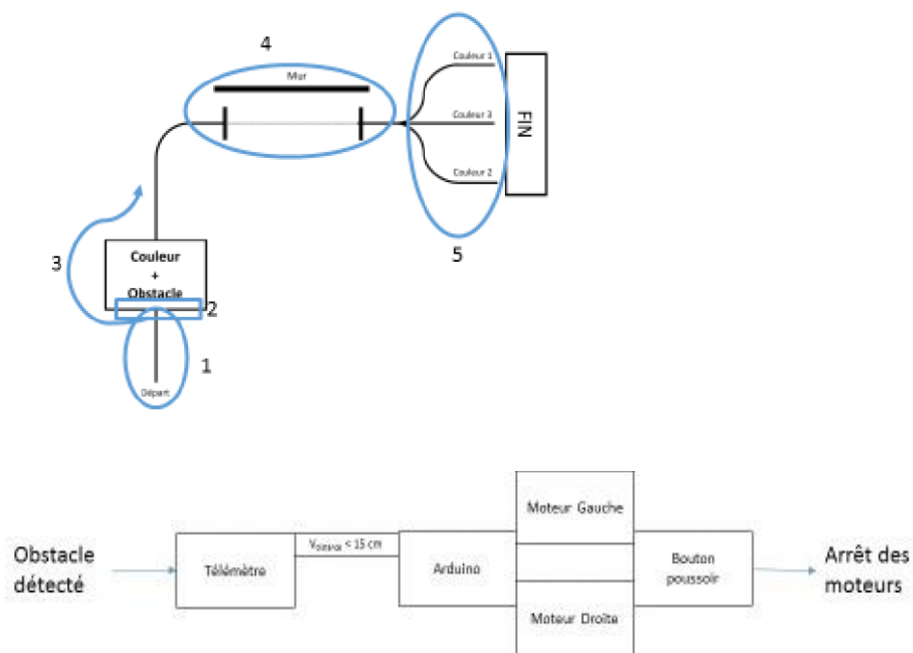
1. Introduction	1
2. Schéma fonctionnel	3
3. Fonctions	4
3.1 – Suivideligne()	5
3.2 – Avancer()	7
3.3 – TourDroite() / TourGauche()	8
3.4 – Telemetre()	10
3.5 – Boutonpoussoir()	12
3.6 – Capteurcouleur()	12
3.7 – EsquiveObstacle()	14

2 sur 28

3.8 – Suividemur	15
3.9 – Servomoteur	16
3.10– Chemincouleur()	17
4. Planning	19
5. Validation du projet	20
6. Conclusion	21
7. Annexes	22

2. Schéma fonctionnel

Voici un exemple de la piste qui sera présente lors de la validation du projet. Elle contient toutes les taches que devra effectuer le robot de son point de départ à son point d'arrivé. Ces tâches ont été numérotées et ensuite expliquées à l'aide de schéma fonctionnel.

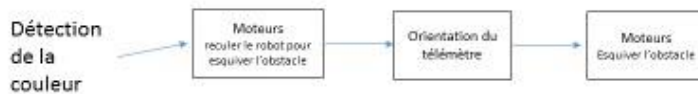


Fonction 1 : Détection d'obstacle

Ce schéma représente la première fonction qui sera effectuée dans ce SAE. Elle consiste en la détection de l'obstacle jusqu'à que l'on arrête à l'aide du boutonpoussoir

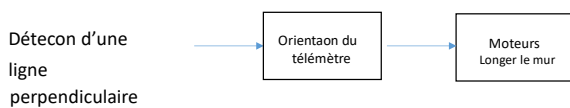


Fonction 2 : Récupération de la couleur de l'obstacle



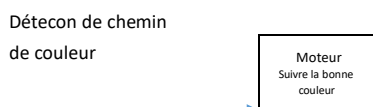
Fonction 3 : contournement de l'obstacle

Le schéma suivant représente le contournement de l'obstacle qui sera réaliser à l'aide de l'orientation du télémètre



Fonction 4 : Détection d'une ligne perpendiculaire

Ce schéma représente la détection d'une ligne perpendiculaire et ensuite le passage du robot le long du mur.



Fonction 5 :

Ce schéma représente la fonction qui permettra de détecter la couleur.

3. Fonctions

Voici les différentes fonctions que nous avons créé pour répondre aux besoins de chaque étape :

- `suivideligne()`
- `capteurcouleur()`
- `boutonPoussoir()`
- `telemetre()`
- `avancer()`
- `tourDroite()`
- `tourGauche()`
- `EsquiveObstacle()`
- `SuivideMur()`
- `CheminCouleur()`
- `Servomoteur()`

3.1 – Suivideligne()

Inutile de le repréciser, l'une de nos fonctions les plus importantes de ce projet est le suivi de ligne. Étant l'objectif principal du SAE précédent, le suivi de ligne

permet à notre robot de suivre correctement une ligne noire sur un fond clair. Dans notre cas, le suivi de ligne permet au robot de suivre une ligne noire sur un fond plus ou moins blanc.

Contrairement au SAE précédent où nous avons créé notre propre suivi de ligne, dans celui-ci nous avons utilisé le capteur RGB LineFollower qui fera office de nos capteurs précédents.

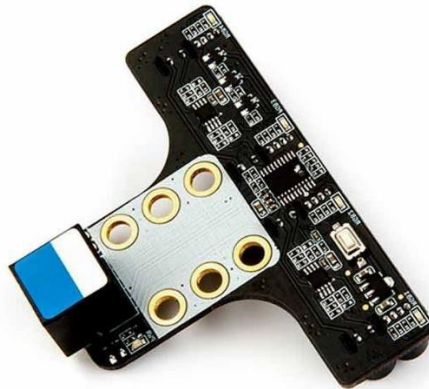


Figure 1 : Capteur RGB LineFollower

Ce capteur se branche sur la carte Arduino sur les broches de communication I2C. Comme au SAE précédent, Il est embarqué lui-même de quatre capteur de couleur. Pour traiter les données du capteur avec arduino nous allons utiliser dans notre code la bibliothèque suivante qui nous permet de l'intégrer :

```
#include "MeRGBLineFollower.h"
```

Figure 2 : Bibliothèque du LineFollower

Nous câblerons le capteur de la manière suivante :

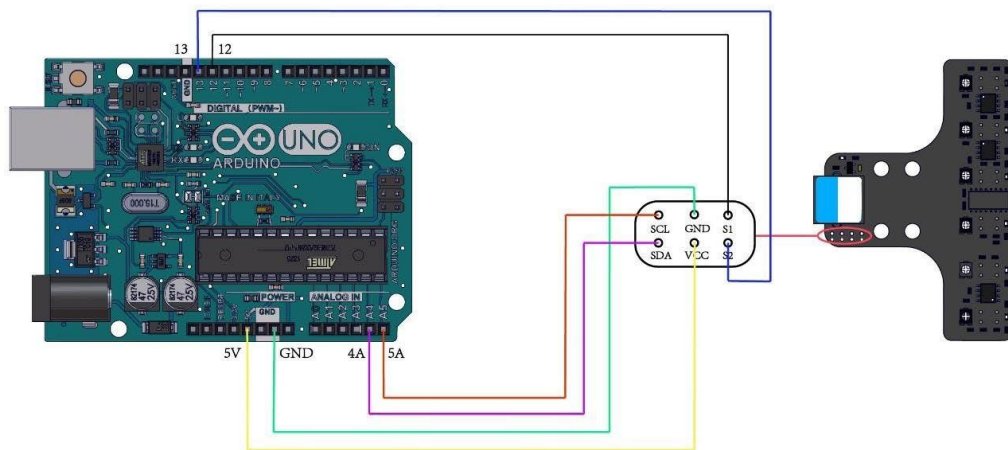


Figure 3 : Câblage du RGB LineFollower

Contrairement à nos capteurs précédents où il fallait déterminer un seuil pour différencier le blanc du noir, celui-ci possède son propre mode d'apprentissage pour différencier la ligne de l'arrière-plan.

Mais le principe est le même : nous récupérons une variable qui varie de 0 à 15, lui indiquant la position du robot par rapport à la ligne.

Relevé sur la ligne	Valeur de turn offset	Fonction à réaliser
○●●○	9	avancer()
●●●●	0	avancer()
○○○○	15	avancer()
○○●●	3	tourDroite()
○●●●	7	tourDroite()
○○○●	11	tourDroite()
●●○○	12	tourGauche()
●●●○	14	tourGauche()
●○○○	13	tourGauche()

Figure 4 : Table de vérité du suivi de ligne

3.2 – Avancer()

La fonction avancer permet au robot comme son nom l'indique d'avancer tout droit. C'est une fonction que nous avons déjà conçu pour le SAE précédent mais il est important d'en reparler ici car nous l'utiliserons à nouveau :

```
void avancer() {  
  
    DDRB = 0xff ;  
  
    DDRD = 0 ;  
  
    TCCR1A = 0xA2 ;  
  
    TCCR1B = 0x1B ;  
  
    ICR1 = 12500 ;  
  
    OCR1A = 800; // Moteur droit  
  
    OCR1B =800; // Moteur gauche }
```

Cette fonction permet d'alimenter les moteurs avec des TIMERS. Ici nous utiliserons le TIMER1 qui permet au moteur d'avancer tout droit.

3.3 – TourDroite() / TourGauche()


```
void tourDroite() {  
  
    DDRB = 0xff ;  
  
    DDRD = 0 ;  
  
    TCCR1A = 0xA2 ;  
  
    TCCR1B = 0x1B ;  
  
    ICR1 = 12500 ;  
  
    OCR1A = 1400; // Moteur droit  
  
    OCR1B = 0; // Moteur gauche  
  
}
```

La fonction tourDroite permet de diriger le robot vers la droite en jouant sur l'extinction du moteur gauche (c'est la raison pour laquelle OCR1B est à 0). C'est une fonction que nous avons déjà réalisée pour le SAE précédent.

```
void tourGauche() {  
  
    DDRB = 0xff ;  
  
    DDRD = 0 ;  
  
    TCCR1A = 0xA2 ;  
  
    TCCR1B = 0x1B ;  
  
    ICR1 = 12500 ;  
  
    OCR1A = 0; // Moteur droit  
  
    OCR1B = 1400; // Moteur gauche  
  
}
```

La fonction tourGauche permet de diriger le robot vers la gauche en jouant sur l'extinction du moteur droit (c'est la raison pour laquelle OCR1A est à 0). C'est une fonction que nous avons déjà réalisée pour le SAE précédent.

3.4 – Telemetry()

La fonction télémètre nous permet de récupérer le signal de sortie du télémètre. C'est ce qui nous permet de récupérer en bit une valeur en fonction de la distance en utilisant le Convertisseur Analogique Numérique (CAN).

```
// Fonction de lecture de la tension sur la broche A1
uint16_t readADC() {
    // Démarrage de la conversion
    *ADCSRA_REG |= (1 << 6);

    // Attente de la fin de la conversion
    while (*ADCSRA_REG & (1 << 6));

    // Lecture des valeurs ADC (16 bits)
    uint16_t adcValue = *ADC_REG;

    return adcValue;
}
```

Figure 5 : Convertisseur Analogique Numérique

Voici une courbe qui représente la sortie :

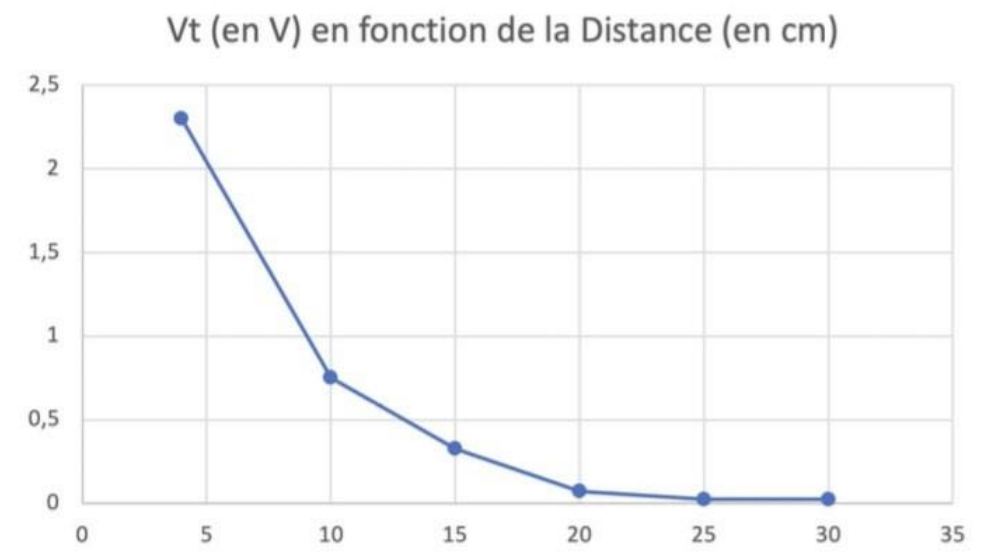


Figure 6 : Courbe de télémètre

Comme nous pouvons le constater, la tension de sortie diminue grandement en fonction de la distance. Afin d'utiliser la fonction, nous allons à de nombreuses

reprises comparer la valeur du télémètre pour donner des instructions au robot. Par exemple, si l'on récupère la valeur du signal de sortie du télémètre à 15cm, il est possible de lui demander de ralentir, d'accélérer, de s'arrêter, etc. À partir de cette distance précise.

3.5 – Boutonpoussoir()



Figure 7 : Bouton poussoir

Notre fonction Boutonpoussoir permet de récupérer l'entrée numérique de l'entrée A0. Notre entrée A0 est en réalité la sortie du bouton poussoir qui nous permet de savoir si celui-ci est activé ou non. Nous utilisons la fonction dans le cas présent :

```
int boutonpoussoir() {  
    int etatbp = digitalRead(A0);  
    return etatbp;  
}
```

Figure 8 : Fonction Boutonpoussoir()

Si le bouton poussoir est à l'état haut, nous donnerons au robot des instructions à réaliser.

3.6 – Capteurcouleur()

Pour récupérer une couleur, nous avons à notre disposition le capteur de couleur TCS 34725.

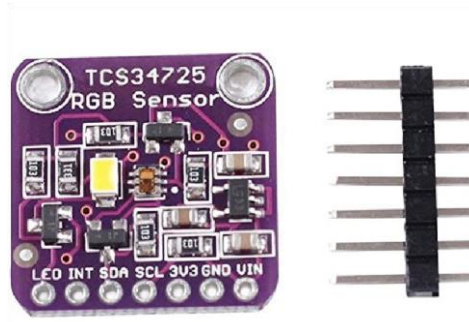


Figure 9 : Capteur de couleur TCS34725

Le câblage du capteur, quant à lui, est assez facile :

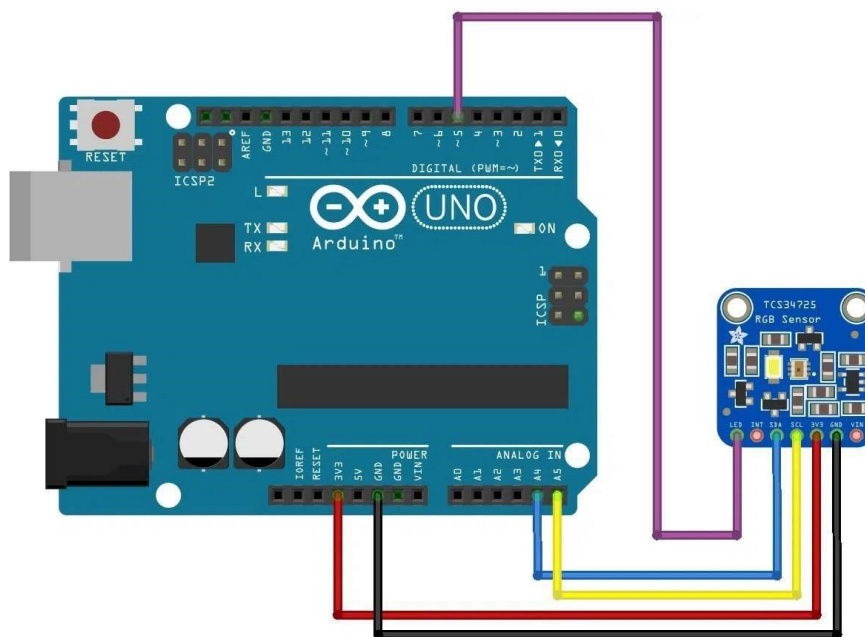


Figure 10 : Câblage du TCS34725

La fonction Capteurcouleur permet de récupérer une couleur de la surface devant lui. Cette fonction renvoie la couleur sous la forme RGB (Rouge Vert Bleu) sous un résultat de 16 bits. Afin que cette fonction soit plus facile à exploiter, nous avons de notre propre initiative mis des seuils de couleur afin de récupérer une des trois couleurs primaires (Par exemple, si le seuil de la couleur rouge est supérieur aux deux autres, on récupère la couleur Rouge)

Dans notre cas, nous avons introduit une variable **couleur** qui a trois valeurs possibles :

- Si rouge ; couleur = 1
- Si vert ; couleur = 2
- Si bleu ; couleur = 3

```
void Couleur() {
    uint16_t r, g, b, c;
    tcs.setIntegrationTime(TCS34725_INTEGRATIONTIME_614MS);
    tcs.getRawData(&r, &g, &b, &c);
    if ((r > g) && (r > b) && (r > 200)) {
        couleur = 1;
        digitalWrite(redpin, HIGH);
        digitalWrite(greenpin, LOW);
        digitalWrite(bluepin, LOW);
        RGBLineFollower.setRGBColour(RGB_COLOUR_RED);
        Serial.println("C'est rouge");
    } else if ((g > r) && (g > b) && (g > 100)) {
        couleur = 2;
        digitalWrite(greenpin, HIGH);
        digitalWrite(redpin, LOW);
        digitalWrite(bluepin, LOW);
        RGBLineFollower.setRGBColour(RGB_COLOUR_GREEN);
        Serial.println("C'est vert");
    } else if ((b > r) && (b > g) && (b > 100)) {
        couleur = 3;
        digitalWrite(bluepin, HIGH);
        digitalWrite(redpin, LOW);
        digitalWrite(greenpin, LOW);
        RGBLineFollower.setRGBColour(RGB_COLOUR_BLUE);
        Serial.println("C'est bleu");
    }
}
```

Figure 11 : Fonction Capteurcouleur()

3.7 – EsquiveObstacle()

La fonction EsquiveObstacle() permet au robot , dans les grands termes, d’esquiver un obstacle devant-lui. Elle utilise la fonction telemetre() pour indiquer au robot quoi faire au vu de l’obstacle.

Il existe deux fonctions possibles à cette fonction :

- Si le robot récupère une valeur du télémètre inférieur à un seuil fixé (si un obstacle est proche) , le robot **avance**
- Si le robot récupère une valeur du télémètre supérieur à un seuil fixé (si l'obstacle est loin), le robot **tourne à gauche**

```
void dodge() {
    telemetregauche();
    int T1 = Telemetre1();
    while ((T1 >= 200)) {
        avancer();
        Serial.println("Obstacle, avance tout droit");
        T1 = Telemetre1();
    }
    tourGaucheObstacle();
    Serial.println("Plus d'obstacle, tourne à gauche");
    T1 = Telemetre1();
}
```

Figure 12 : Fonction EsquiveObstacle()

Ces deux scénarios permettent au robot de contourner l'obstacle sans grande difficulté.

3.8– Suividemur

La fonction Suividemur() nous permet de « compléter » le suivi de ligne. Après l'esquive de l'obstacle et une fois la ligne noire non détectée, le robot devra orienter le télémètre à sa gauche afin de longer le mur à une distance convenable. Pour réaliser

cela, cette fonction fait appel à plusieurs fonctions tel que la fonction Telemetre() et la fonction Servomoteur().

```
void SuiviMur() {
    telemetregauche();
    int T1 = Telemetre1();
    while ((T1 >= 250)) {
        avancer();
        Serial.println("Obstacle, tourne droite");
        T1 = Telemetre1();
    }
    tourGaucheObstacle();
    Serial.println("Plus d'obstacle, tourne à gauche");
    T1 = Telemetre1();
}
```

Figure 13 : Fonction SuiviMur

3.9– Servomoteur

La fonction Servomoteur() fait appel au TIMER2, pour diriger le servomoteur du télémètre. Ici, nous avons fixé 3 positions pour notre servomoteur :

- Servomoteur Gauche
- Servomoteur Centre
- Servomoteur Droit

Ces trois positions sont fixés à l'aide du OCR2A.

Pour trouver la position qui nous convenait, nous avons intentionnellement tester différentes valeurs pour OCR2A afin de déterminer celles qui nous convenait le plus.

```
void telemetredroite() {  
    OCR2A = 50;  
}  
  
void telemetregauche() {  
    OCR2A = 300;  
}  
  
void telemetrecentre() {  
    OCR2A = 185;  
}
```

Figure 14 : Fonctions Servomoteur

3.10– Chemincouleur()

Notre dernière fonction Chemincouleur() réalise la dernière étape de notre projet : Elle utilise la couleur déduite de la fonction CapteurCouleur() afin de déterminer le chemin à prendre à la dernière étape du projet

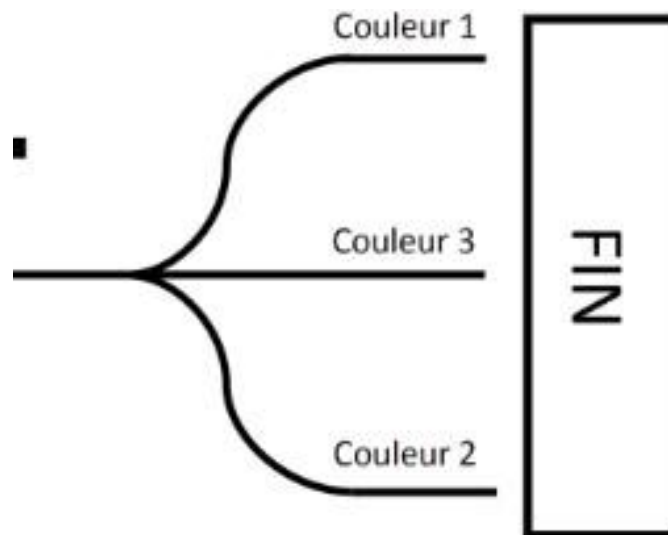


Figure 15 : Chemin de fin de parcours

À l'aide de la couleur, la fonction indiquera au robot le chemin à prendre afin de finaliser le projet.

Par exemple, si le robot récupère la couleur bleue, alors il suivra le chemin dédié à la couleur tout en s'arrêtant à une distance raisonnable de l'obstacle de fin de parcours.

```

void CheminCouleur() {
  if (couleur == 1) { // Tourne à gauche
    tourGauche();
    delay(5000);
    Suivi();
    Serial.println("Chemin gauche");
  }
  else if (couleur == 2) { // Tourne à droite
    tourDroite();
    delay(5000);
    Suivi();
    Serial.println("Chemin droite ");
  }
  else if (couleur == 3) { // Avancer tout droit
    avancer();
    delay(5000);
    Suivi();
    Serial.println("Tout droit");
  }
}

```

Figure 17 : Fonction CheminCouleur()

4. Planning

Voici notre planning qui nous servira tout au long du projet sur nos fonctions et tâches à réaliser. Celui-ci est temporaire et n'est pas totalement indicatif de quand nous allons réellement réaliser chaque fonction. 1^{er} planning

	19/03		20/03		27/03		03/04		03/04		22/04		24/04		28/04		30/04	
Fonctions	2h	4h	2h	4h	2h	4h	2h	4h	2h	4h	2h	4h	2h	4h	2h	4h	2h	4h
Fonctions																		
Planning																		
Modification robot SAE3																		
Fonction 1 : Récupérer couleur																		
Fonction 2 : Obstacle(Télémètre)																		
Fonction 3 : Contournement obstacle																		
Fonction 4 : Ligne perpend																		
Fonction 5 : Détection couleur																		
Test + ajustements																		
Code complet à revisiter																		
Test complet																		

2nd planning

	22/04		23/04		24/04		29/04		30/04		10/06		10/06	
Fonctions	2h	4h	2h	4h	2h	4h	2h	4h	2h	4h	2h	4h	2h	4h
Seuil couleur ROB + CAN TELEMETRE														
Reculer le robot de l'obstacle														
Éviter l'obstacle et retour sur la ligne														
Pivoter le bras du télémètre														
Suivre des chemins de couleur														
Test de l'ensemble du robot														
Concours														

Figure 18 : Planning du projet

Nous avons réussi à réaliser toutes les fonctions que nous nous étions fixés lors de ce planning.

5. Validation du projet

Pour la validation du projet, nous avons participé à un concours qui avait pour but de tester les robots de chaque groupe sur une piste spécialement conçue pour évaluer différentes fonctionnalités.

Malheureusement, notre robot n'a pas bien fonctionné pour de nombreuses raisons :

- Problèmes de code : Des erreurs dans notre façon de concevoir le programme du robot ont affecté son utilité.
- Problèmes des moteurs : Les moteurs n'étaient pas assez puissants, affectant la vitesse , la fluidité et la précision des mouvements.
- Capteurs lents : Les capteurs n'étaient pas assez rapides pour envoyer les données en temps réel, causant des erreurs sur la détection de ligne après l'esquive de l'obstacle.

Pour ces diverses raisons, il est difficile de considérer ce projet comme validé en raison des nombreux problèmes rencontrés. Néanmoins, ce projet reste un sujet intéressant car il nous a permis de mieux comprendre les défis de la robotique mais aussi de la programmation et d'identifier des domaines clés à améliorer pour nos futurs développements.

6. Conclusion

En conclusion, le projet durant ces deux semestres de création d'un robot suiveur de ligne a été une expérience intéressante et réussie. Nous avons fabriqué un robot capable de suivre une ligne tracée au sol, d'éviter des obstacles, de suivre un mur et de choisir des chemins en fonction des couleurs détectées.

Ce projet nous a permis de pratiquer différentes compétences en programmation, en électronique et en résolution de problèmes. Grâce à une bonne planification et à de nombreux tests, nous avons pu surmonter les défis techniques et atteindre nos objectifs.

Au final, nous avons un robot autonome efficace qui remplit plus ou moins les tâches demandées.

7. Annexes

